



BLOOM-FaaS: Early Filtering For Serverless Joins Using Distributed Bloom Filters

Phan-An-Truong Tran, Laurent d'Orazio, Le Gruenwald, Thuong-Cang Phan

► To cite this version:

Phan-An-Truong Tran, Laurent d'Orazio, Le Gruenwald, Thuong-Cang Phan. BLOOM-FaaS: Early Filtering For Serverless Joins Using Distributed Bloom Filters. 2026. <hal-05722983>

HAL Id: hal-05722983

<https://hal.science/hal-05722983v1>

Preprint submitted on 21 Aug 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Copyright - All rights reserved

BLOOM-FaaS: Early Filtering For Serverless Joins Using Distributed Bloom Filters

Phan-An-Truong Tran
Univ Rennes, CNRS, IRISA
Rennes, France
truong.tran@irisa.fr

Laurent d’Orazio
Univ Rennes, CNRS, IRISA
Rennes, France
laurent.dorazio@univ-rennes1.fr

Le Gruenwald
School of Computer Science
The University of Oklahoma
Norman, OK, USA
ggruenwald@ou.edu

Thuong-Cang Phan
Can Tho University
Can Tho, Vietnam
ptcang@cit.ctu.edu.vn

Abstract—Serverless computing provides a deployment model that eliminates infrastructure management while adopting a pay-as-you-go pricing model. Query-as-a-Service systems are particularly suitable for workloads with infrequent access patterns and highly dynamic execution requirements. However, existing state-of-the-art approaches remain limited in their ability to eliminate redundant data during query processing. This issue is especially critical for cloud-based storage systems, where costs are directly tied to the number of read and write operations. In this paper, we propose BLOOM-FaaS, a distributed query processing framework that integrates early-stage filtering with Bloom Filters in a serverless environment. BLOOM-FaaS eliminates unnecessary data in the early stages of the processing pipeline, reducing intermediate data and the number of requests to cloud-based storage systems and decreasing function invocations, ultimately optimizing both execution time, monetary cost, and overall query latency. We evaluate BLOOM-FaaS using the TPC-H benchmark with 330 generated queries. Experimental results show that BLOOM-FaaS reduces intermediate data by 63.46%, function invocations by 51.1%, estimated monetary cost by 55.3%, and total object storage requests by 56.1%, while improving execution time by up to 26.2% compared to state-of-the-art approaches.

Index Terms—Serverless computing, Function-as-a-service, Bloom filter, Early filtering

I. INTRODUCTION

In recent years, the rapid development of cloud computing has fundamentally changed the way large-scale data is stored and analyzed. Instead of maintaining a fixed server cluster with high costs and limited scalability, organizations are increasingly shifting to cloud-based data processing models, in which resources are provided on demand and billed based on actual usage [13]. A prominent trend in this context is Query-as-a-Service (QaaS), such as Athena [6] and BigQuery [20], which allows users to execute analytical queries directly on data stored in object storage without deploying or managing traditional systems [27]. QaaS systems leverage columnar data formats (such as Parquet, ORC, etc) combined with the flexible scalability of cloud infrastructure to reduce operational costs and accelerate deployment [27]. However, QaaS is often limited in its ability to optimize complex queries, particularly join operations on large datasets.

Function-as-a-Service (FaaS), such as AWS Lambda [7], Google Cloud Functions [21], and Microsoft Azure Functions [33], has emerged as an attractive computing model for data

analytics, thanks to features such as automatic scaling based on the number of function invocations and execution time [24]. Several systems have proposed leveraging FaaS to build query processing engines directly on object storage, aiming to combine the flexibility of serverless with the demands of large-scale data analytics. However, performing join operations in serverless environments remains a significant challenge [27]. FaaS is inherently stateless, which poses a significant challenge for event-driven distributed data processing. Since FaaS functions cannot retain state between invocations, intermediate data must be written to and read from external storage systems such as object storage, key-value stores, or network file systems. This introduces additional I/O overhead, increases end-to-end latency, and raises the overall cost of the processing pipeline.

In this paper, we propose BLOOM-FaaS, a serverless join framework that performs distributed early filtering tailored to FaaS constraints. Each function builds a local Bloom filter on its partition; these are merged through a low-latency short-term storage layer into a global filter that prunes non-matching records before shuffle, reducing intermediate data, I/O cost, execution time, and overall monetary cost.

In summary, the paper makes the following contributions:

- We present BLOOM-FaaS, a serverless join that introduces a distributed Bloom filter architecture combining an adaptive coordinator, two-tier shuffle storage, and multi-level Bloom filter pushdown to significantly reduce shuffle overhead, object storage requests, overall processing time, and monetary cost.
- We propose early filtering, a stateless architecture for distributed Bloom filter construction in serverless environments, featuring metadata callbacks and a self-calibration mechanism for automatic Bloom filter sizing.
- We evaluate the performance and effectiveness of early filtering and compression codes in BLOOM-FaaS compared with state-of-the-art systems in terms of query execution time, storage overhead, and function invocation, and overall monetary cost.

The remainder of this paper is organized as follows. Section II positions BLOOM-FaaS against prior work, and Section III reviews the necessary background. Section IV describes the

BLOOM-FaaS design, Section V presents the evaluation, and Sections VI discuss findings and conclude.

II. RELATED WORK

This section reviews related work on join query processing in serverless environments, focusing on four topics: serverless query engines, early filtering with Bloom filters, shuffle services, and estimating the dollar cost of serverless computing. Because functions are stateless, every intermediate result must be written to and read back from object storage, which is billed per request and per byte; a tuple that is shuffled but never matches is cost paid for nothing.

Serverless query engines: Starling [36], Lambada [35], and Skyrise [15] are notable studies on systems that leverage Function-as-a-service to process data on external storage (e.g., object storage, key-value stores, etc). These works primarily address the challenge of optimizing read/write costs when using S3 [3] as an intermediate data exchange layer during join operations, employing techniques such as intermediate combiner [36], two-level exchange [35], and two-level invocation [35]. However, these techniques reduce the per-request overhead of the exchange rather than its volume: without early filtering, non-matching tuples are paid for on both the write and the read path.

Early filtering with Bloom filter: Bloom filter (BF) [14] and BloomJoin [16], [30] are efficient techniques for checking the existence of join keys and eliminating redundant data early with low computational cost. In traditional distributed systems, several studies such as Intersection BF [37], Adaptive Intersection BF [38], and PushdownDB [40] have employed BF to filter out non-matching keys before executing join operations. A common approach is to build a local BF for each data partition and merge it into a global filter representing the entire dataset. These methods attack intermediate-data volume at its source, but were designed for serverful systems and have not been explored in serverless environments, where limitations in state management, communication, and intermediate data handling raise significant challenges.

Shuffle service for serverless: Serverless computing offers strong burst scalability within short time frames [15], but faces major limitations in join operations due to the stateless nature of functions. The inability to directly share intermediate data forces systems to rely on external storage for shuffle operations, resulting in high cost and latency. To address this issue, studies such as CloudBurst [39], Pocket [26], and Jiffy [25] have proposed shuffle services or low-latency intermediate memory layers based on server architectures, enabling more efficient data exchange between serverless functions. However, they accelerate the transfer through a provisioned serverful layer that is itself billed for the query duration, without reducing the volume of intermediate data.

Estimating the dollar-cost of serverless computing: Public cloud providers offer transparent pricing models that allow users to estimate the cost of their resource consumption. Sizeless [19] and PRCP [29] evaluate the performance of big data processing using open-source Function-as-a-Service

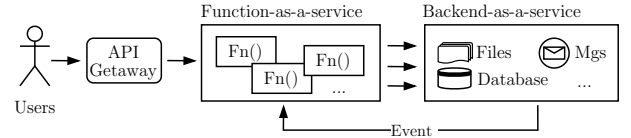
(FaaS) platforms deployed on top of Kubernetes (K8s), and provide serverless cost estimation with low prediction error. AWS EMR [4] adopts a simple and predictable pricing model, in which users are charged on a per-second basis according to the resources consumed, making the overall processing cost relatively straightforward to estimate.

From the above analysis, existing systems reduce per-request overhead and accelerate the transfer, but none reduces the number of non-matching tuples crossing object storage. To address this, this study proposes BLOOM-FaaS, a framework that applies BF-based early filtering to eliminate records that do not participate in the join before they are materialized in object storage.

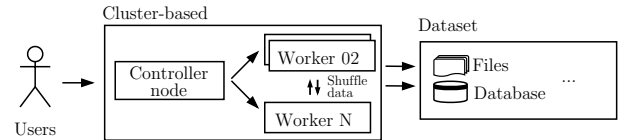
III. BACKGROUND

A. Function-as-a-Service

Function-as-a-Service (FaaS) follows an event-driven model (See Figure 1a), where each function is triggered by specific events such as HTTP requests, data streams, or events from Backend-as-a-Services (like when a file is uploaded to AWS S3, a row is inserted into DynamoDB, etc) [24]. Each function instance is subject to strict constraints on execution timeout, number of vCPUs, and RAM capacity. The pricing model is based on resource consumption rather than the fixed hourly/monthly cost of traditional clusters. For unpredictable workloads, FaaS is particularly suitable due to its burst scaling capability (from zero to thousands of parallel instances within seconds) [12], [34] and instant resource deallocation after task completion, which is an advantage over machine-based models. In contrast, traditional clusters (See Figure 1b) require controller and worker nodes to run continuously with fixed cost.



(a) Architecture of a Function-as-a-Service (FaaS) model, where stateless functions are dynamically invoked by events through an API gateway and interact with backend services.



(b) Architecture of a cluster-based computing model, where a controller node manages multiple worker nodes with fixed resources and supports internal data shuffling for distributed processing.

Fig. 1: Architectural comparison between cluster-based computing and Function-as-a-Service (FaaS).

B. Serverless Storage

Cloud platforms offer different storage models to meet needs for speed, cost, and scalability. There are four main

TABLE I: Pricing of AWS Serverless Storage Services

Services	Storage (\$/GiB-mo)	Requests (\$/M)		Transfer (\$/GiB)	Latency (ms)
		Read	Write		
S3 Standard	0.021–0.023	0.0053	0.0053	0	100–200
DynamoDB	0.25	0.25	1.25	0	1–10
ElastiCache ^a	105.1	^b	^b	0	< 1
EFS	0.17–0.3	0	0	0.01	0.2–3

^a Redis OSS engine; storage cost is the amortized monthly node price.

^b Requests are not billed separately. Assuming 1 operation $\approx$ 1 KiB (0.05–0.1 ECPU), the equivalent cost is \$0.0002–\$0.0004 per million operations.

types of storage [15]: object storage [3], [31], key-value storage [5], [22], in-memory storage [2], [23] and distributed file system [1], [32]. Object storage, such as AWS S3 [3] and Azure Blob Storage [31], is designed to store immutable binary objects of varying sizes and to provide scalable bandwidth for accessing these objects. Key-value stores, such as DynamoDB [5] and GCP Firestore [22], support lower latency key lookups at higher IOPS for kilobyte-sized values. In-memory stores, such as AWS ElastiCache and GCP Memorystore, support lower microsecond latency. AWS EFS, Azure Files, and other networked file systems with an NFS or SMB interface provide an abstraction of files and directories [15]. Table I presents the differences in storage costs, access pricing models, data transfer costs, and latency for common storage types on AWS infrastructure. It can be seen that object storage offers low storage costs but high latency and non-negligible read/write costs, while in-memory storage provides very low latency but at a high storage cost.

Query-as-a-Service (QaaS) or serverless data processing model that can query data directly from object storage without requiring datasets to be pre-registered in a persistent centralized metadata catalog (catalog-less) [27]. Intermediate data in processing pipelines is often temporarily stored in cloud storage services [15], [35], [36], most commonly object storage. However, object storage uses a pricing model based on the number of read/write operations, which significantly increases costs when pipelines have many intermediate processing steps and large shuffle data volumes. In complex workflows, these costs can grow rapidly and become difficult to control.

C. Bloom Filter

Bloom filter (BF) [14] is a probabilistic data structure used to compactly represent a set. With n inserted elements, a standard BF is a single array of m bits and k independent hash functions that map elements to positions in the range $[0, m - 1]$ in the array. When inserting an element, each of the k functions is used to produce an index, and the corresponding bit is set to 1. When querying, an element is considered present if all bits in the positions given by the k hash functions are set to 1. Based on the n elements and the False Positive Rate (ϵ), Formula 1 [14] determines the size m of the bit array. To determine the optimal number of hash functions k , we use

Formula 2 [14], and the False Positive Rate can be calculated using Formula 3 [14].

$$m = -\frac{n \ln \epsilon}{(\ln 2)^2} \quad (1) \quad k = \frac{m}{n} \ln 2 \quad (2)$$

$$\epsilon = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k \quad (3)$$

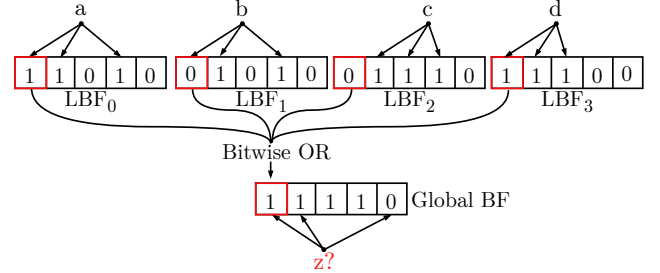


Fig. 2: A partitioned BF with four partitions is processed independently by four workers. Each worker constructs a local BF ($LBF_0, LBF_1, LBF_2, LBF_3$), these local BF's are then merged into a global BF using a bitwise OR operation. The element z is subsequently queried to determine whether it is present in the set.

Columnar formats such as Parquet and ORC are widely used for cloud-based data processing because they maintain metadata that supports predicate and projection pushdown [28]. They also provide native Bloom-filter mechanisms associated with columns or data blocks [9], [10]. These BF's are typically constructed when data is written and therefore reflect the physical organization of the stored data. In contrast, predicates in analytical workloads are determined by queries submitted at runtime. A runtime-generated BF can therefore complement such static file-level metadata by adapting the filtering structure to the keys and predicates involved in a particular query. In distributed data processing, the input set may be partitioned across multiple workers, with each worker having access only to its local partition. A common approach is therefore to construct a local Bloom filter (LBF) independently for each partition and subsequently combine the local filters into a global Bloom filter [37], [38]. As illustrated in Figure 2, join keys such as a, b, c , and d are distributed across different workers. For an element x , each worker applies the same set of k hash functions, and each hash value is mapped to a bit position as follows:

$$\text{index} \leftarrow h_i(x) \bmod m, \quad i = 1, \dots, k \quad (4)$$

where x is the element to insert, $h_1, \dots, h_k$ are the hash functions, and m is the BF length in bits. The worker then sets the corresponding bits in its local BF's (e.g., $LBF_0, LBF_1, \dots$). Once all local BF's are constructed, they are merged into a global BF by performing a bitwise OR on corresponding bit positions (e.g., $\text{BF}[i] = LBF_0[i] \vee LBF_1[i] \vee \dots \vee LBF_n[i]$). textit is considered a member if all positions in the global

BF, obtained by applying the same k hash functions, are set to 1.

IV. PROPOSED BLOOM-FAAS ARCHITECTURE

A. Overview

Figure 3 provides an overview of the BLOOM-FaaS architecture, our proposal consists of three layers: (1) the Adaptive Coordinator acts as the control plane, compiling query pipelines into directed acyclic graph (DAG), orchestrating execution stages, and dynamically adjusting parallelism based on runtime feedback; (2) the Compute Layer executes each DAG stage as stateless FaaS functions, where each function embeds a lightweight SQL engine with BF support; and (3) the Serverless Storage layer is divided into two tiers: long-term storage for input data and final outputs, and short-term storage for intermediate data and BFs exchanged among functions.

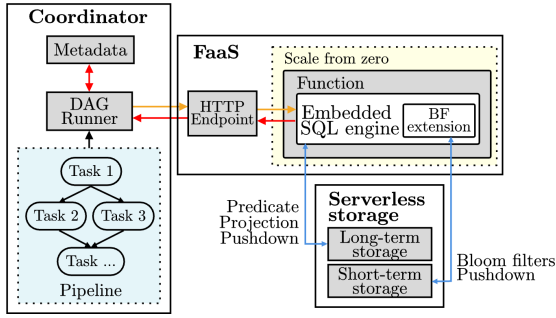


Fig. 3: BLOOM-FaaS architecture. The three layers exchange information through two control-plane channels. Yellow arrows carry function invocations from the DAG Runner to FaaS endpoints, and red arrows carry metadata feedback back to the Coordinator. Blue arrows show data-plane traffic between functions and storage.

B. Adaptive Coordinator

The Adaptive Coordinator serves as the controller for BLOOM-FaaS, comprising 3 components: a DAG Runner, a Metadata callback, and an Adaptive decision logic.

DAG Runner. It receives the pipeline from the users, compiles the pipeline into a DAG, and orchestrates execution across the compute layers. It triggers independent steps in parallel and dependent ones sequentially. The runner also tracks the status of each step and retries or reschedules invocations on failure.

Metadata callback. Each invocation returns a metadata response to the Coordinator summarizing the output size, row count, intermediate storage location, and the BF ratio, etc. This is the only feedback channel that the Coordinator uses to tune downstream steps.

Adaptive decisions. The Coordinator uses metadata to address two questions for each next step: (1) how many function instances to invoke, and (2) how to partition the workload across them. The instance count scales with intermediate data size, more instances for large inputs and fewer for small ones to avoid unnecessary overhead. When upstream outputs are

skewed in size, the Coordinator inserts an additional repartition or aggregation step to balance the load and mitigate stragglers. For scan steps that use BFs, the observed fill ratio also drives self-calibration of filter size for the next run (Bloom filters management will be detailed in Section IV-E).

C. Compute Layer

The compute layer executes each DAG step as stateless functions, with each Coordinator request processing one data partition independently. This design exploits two FaaS properties: (1) burst scaling from zero to thousands of instances without pre-provisioning the cluster, and (2) event-driven invocation based on runtime progress. Unlike systems that separate BF construction and application, BLOOM-FaaS integrates local BF construction, membership checking, and predicate evaluation within each function. Since stateless functions cannot merge local BFs in memory, BF construction and merging use a shared short-term storage layer (Section IV-E). BLOOM-FaaS turns this constraint into an integrated design by pushing BF filtering into each function's I/O layer, eliminating non-matching rows before shuffle.

BLOOM-FaaS provides six operators, and each DAG step maps to one of them. SCAN applies a three-layer pushdown on every row group: projection reads only the required columns, predicate skips row groups whose min/max metadata cannot satisfy the WHERE clause, and BF pushdown discards rows whose join key is absent from the global BF before shuffle (Figure 4). The other five are BF build & merge (builds local BFs and merges them into a global BF through short-term storage), COMBINE (consolidates small intermediate files to reduce downstream I/O), HASH PARTITION, JOIN (broadcast and partitioned), and AGGREGATE (SUM, COUNT, AVG, GROUP BY, HAVING).

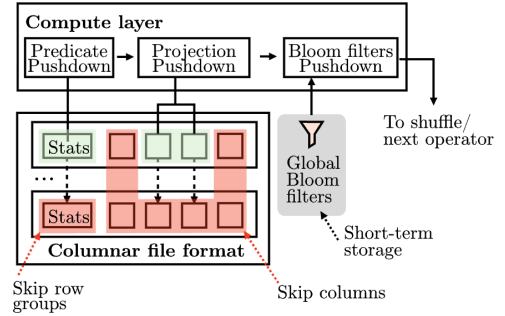


Fig. 4: Three-layer pushdown applied by the scan operator on a row group

D. Serverless Storage

The storage layer is the only exchange channel between functions in BLOOM-FaaS since FaaS functions do not share memory and intermediate data must travel through external storage. This constraint imposes two different access patterns: input data requires high bandwidth and long-term durability, whereas intermediate data requires low latency and near-zero

per-request cost. BLOOM-FaaS addresses this by splitting storage into two tiers.

Long-term storage. This tier holds input tables and final query results in an S3-compatible object store. It is optimized for large sequential reads and high durability, matching the access pattern of the scan operator: large columnar files, read once per query, and never modified during a run.

Short-term storage. This tier holds the intermediate data of a single pipeline run, including local BF's produced during fan-out, the global BF produced during fan-in, and filtered partitions passed between steps. It is backed by a shared file system with sub-millisecond latency and no per-request fee (detail in Section III-B). This is the direct enabler of distributed BF aggregation: hundreds of functions write small BF's concurrently, and routing them through object storage would multiply the number of billable requests and dominate query latency.

E. Bloom Filter Construction With Self-Calibration

Because there is no shared memory between functions, BLOOM-FaaS builds the global BF through a fan-out/fan-in topology over the short-term storage layer. Every function mounts this storage at startup and uses it as the only channel for exchanging BF's. Figure 5 shows the topology.

Fan-out phase. The Coordinator assigns row-group chunks of the build-side table to a set of scan functions and triggers them in parallel. Each function reads its chunk, applies the predicate, and inserts the surviving join keys into a local BF held in memory. Because the function is already iterating over the rows for projection and predicate pushdown, building the BF adds no extra read cost. When the function finishes, it writes its local BF to short-term storage and returns the file path and size to the Coordinator.

Fan-in phase. The Coordinator then triggers a smaller set of merge functions. Each merge function reads several local BF's from short-term storage and combines them with a bitwise OR into one larger BF. Because all local BF's share the same size m and the same k hash functions, the OR operation is associative, so the merge can run as a parallel tree-based reduction. Each level of the tree halves the number of BF's until a single global BF remains. The final merge function writes the global BF to short-term storage for the downstream probe-side functions, and returns the fill ratio ρ to the coordinator for self-calibration.

Sizing a BF requires an estimate of the number of distinct elements n that will be inserted. Traditional systems read this estimate from a query planner or a statistics catalog, but FaaS provides neither. The row count from Parquet metadata is also unreliable, because the WHERE predicates filter most tables aggressively before the BF sees the data, and the reduction factor varies a lot across queries.

Serverless query processing is catalog-less, BLOOM-FaaS solve that with a feedback loop driven by the fill ratio of the merged global BF. After each merge, the system counts the number of set bits c in the bit array of size m . Following [17],

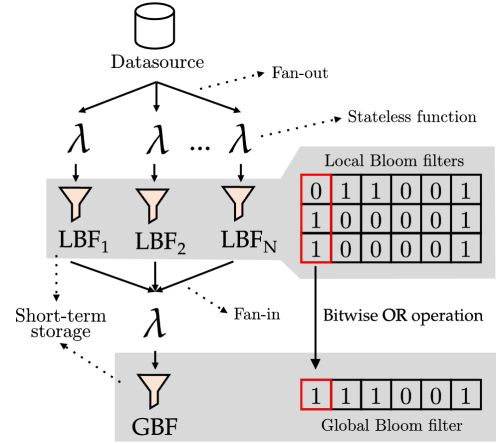


Fig. 5: Overview of distributed BF construction in BLOOM-FaaS. N stateless functions build local BF's in parallel, which are then merged into a global BF via bitwise OR operations, and all intermediate BF's are exchanged through short-term storage.

the number of distinct inserted elements $\hat{n}$ is estimated from the observed fill ratio $\rho = c/m$ as:

$$\hat{n} \approx -\frac{m}{k} \times \ln\left(1 - \frac{c}{m}\right) \quad (5)$$

The estimate is then scaled by ρ to produce n_{target} , the element count used to size the BF for the next run:

$$n_{\text{target}} = \frac{\hat{n}}{\rho} \quad (6)$$

The false positive rate is approximated as $\varepsilon \approx \rho^k$ [18]. For example, at $\rho = 0.95$ with $k = 10$ hash functions, $\varepsilon \approx 60\%$, so six out of every ten non-matching rows incorrectly pass the filter, which nullifies the pruning benefit. At $\rho = 0.5$, ε falls below 0.1%. The first run of a new pipeline uses a conservative default for the element count, and subsequent runs converge to the correct size after a single warm-up. The same fill ratio is also what the adaptive Coordinator uses to determine the parallelism for the next step. Self-calibration of BF size and adaptive fan-out width thus shares a single feedback channel: one observation per merge updates both configurations (as detailed in section IV-B).

V. EVALUATION

In this section, we evaluate BLOOM-FaaS in terms of execution time, resource cost, and filtering effectiveness. We further analyze the impact of early filtering on intermediate data, function invocations, and storage requests, as well as the effects of short-term storage, Parquet compression codecs and estimate monetary cost. Finally, we identify the conditions under which early filtering is most beneficial in serverless environments.

A. Experimental setup

All experiments are conducted on a private cluster deployed on Grid'5000 [11], which provides a high-performance and

configurable environment for evaluating distributed systems. Using Grid'5000 allows us to closely emulate a public cloud setting while maintaining full control over system configuration and ensuring experimental reproducibility across multiple runs.

This scenario evaluates processing time, storage metrics requests, FaaS invocations, the impact of compressed code on overall processing time, and the early filtering effectiveness of BLOOM-FaaS. We compare five systems: (1) Starling-based [36]; (2) Spark on Kubernetes (deployed using the Spark Operator¹) with BF and adaptive execution disabled; (3) Spark with BF and adaptive execution enabled; (4) BLOOM-FaaS using S3 for shuffle; and (5) BLOOM-FaaS FS using a network file system for shuffle.

The experimental environment is deployed on a Kubernetes cluster with 180 vCPU cores, 456 GB of memory, and a 10 Gbps network. The compute layer uses Knative² (v1.21) to provide autoscaling, scale-to-zero, and event-driven execution. Each function is allocated up to 2 vCPU cores and 2 GB of memory, with a maximum of 50 instances. Each function embeds a lightweight SQL execution engine implemented using DuckDB³ (v1.4.4) and extended with a custom BF extension written in Rust⁴, enabling efficient in-memory processing during query execution. For comparison, Spark is configured with up to 50 executors (2 vCPU cores and 2 GB each) and 200 shuffle partitions. The storage layer consists of MinIO⁵ as the long-term object storage and a network file system as the short-term storage for intermediate data (e.g., BFs and partitioned results). Both storage systems are provisioned with 12 vCPU cores, 32 GB of memory, and connected via a 10 Gbps network.

The dataset we are using is the TPC-H⁶ benchmark with scale factor 100 (i.e. the database size is 100 gigabytes), stored in parquet file format, and the QGEN⁷ tool is used to generate 330 random queries from a 22 query template. We focus on 19 representative TPC-H queries (Q2, Q3, Q4, Q5, Q7, Q8, Q9, Q10, Q11, Q12, Q13, Q14, Q15, Q16, Q17, Q18, Q19, Q20, Q21) that involve join operations and varying selectivities, and omit those queries that do not (Q1, Q6, Q22). These 19 queries are chosen to evaluate system performance under diverse query patterns and data characteristics.

B. Processing Time Analysis

This section compares the parallel processing time of five systems. Figure 6 shows the average processing time across 7 runs for five systems on 19 TPC-H queries at a scale factor of 100. BLOOM-FaaS achieves the best performance on 12 out of 19 queries. Overall, it reduces total processing time by 26.6% compared to the Starling-based baseline (the case uses

short-term storage for shuffle data) and by 8.3% compared to Spark AQE, demonstrating that pushing BFs into the FaaS layer effectively.

The largest gains occur in queries that join a selective dimension table with a large fact table. In such cases, BFs built on the dimension table eliminate most fact table rows before the shuffle. Q17 shows the most significant improvement (4.21× over AQE), where shuffle size drops from 9,137 MB to 20.34 MB (450× reduction). Q2 (3.55×) benefits from cascading filters across multiple dimension joins, reducing shuffle from 776 MB to 48.6 MB. Similar patterns are observed in Q8, Q11, Q15, and Q18. In these cases, BLOOM-FaaS with FS slightly outperforms S3 due to lower latency and reduced metadata overhead when repeatedly accessing small filters.

BLOOM-FaaS underperforms Spark AQE on five queries (Q5, Q10, Q13, Q14, Q16) for three main reasons. First, when filters are not selective (e.g., Q5, Q10), they provide limited pruning while incurring FaaS overhead; for instance, Q10 takes 29.43s on BLOOM-FaaS FS versus 14.64s on Spark AQE. Second, for small queries, the cost of constructing and applying BFs outweighs their benefit; Q16 is the worst case (10.23s vs. 2.86s, 3.6x slower), where Spark uses an in-memory broadcast join while BLOOM-FaaS incurs coordination overhead. Q13 exhibits similar behavior.

Spark Serverless on Kubernetes and BLOOM-FaaS differ mainly in their startup model and cold-start behavior. BLOOM-FaaS uses a lightweight 52.67 MB function image and becomes ready in about 0.8-1.2s on warm nodes (or 2-4.5s when the image must be pulled). In contrast, Spark on Kubernetes uses a much larger 720MB executor image and typically needs 18-25s for JVM startup and SparkContext initialization. This result highlights the advantage of the FaaS model for interactive and sporadic workloads.

Overall, BLOOM-FaaS outperforms both Spark variants on average, but its effectiveness depends on query characteristics. It performs best on selective joins and data-intensive workloads, while offering limited benefit for small or low-selectivity queries. The two BLOOM-FaaS variants show comparable overall performance. The FS variant, which uses a network file system for shuffle data, performs better for small, frequently accessed BFs due to lower access latency. In contrast, the default S3-based variant is more efficient for larger data transfers. These results suggest that a cost-based strategy for applying BF pushdown could avoid cases where the overhead outweighs the benefit.

C. Early Filtering Analysis

In a serverless environment, a query is typically executed as a DAG of multiple processing stages. A Bloom filter pushdown (BF pushdown) affects only certain steps, such as fact table scans, combiner partitioning, and intermediate joins. In contrast, several other steps have fixed costs, including dimension table scans (often bounded by infrastructure limits), BF construction, and final aggregation steps. Therefore, to accurately assess the effectiveness of early filtering, the execution time must be divided into two parts: (1) tasks that

¹<https://github.com/apache/spark-kubernetes-operator>

²<https://knative.dev/>

³<https://github.com/duckdb/duckdb>

⁴<https://github.com/duckdb/duckdb-rs>

⁵<https://www.min.io/>

⁶<https://www.tpc.org/tpch/>

⁷<https://github.com/databricks/tpch-dbgen>

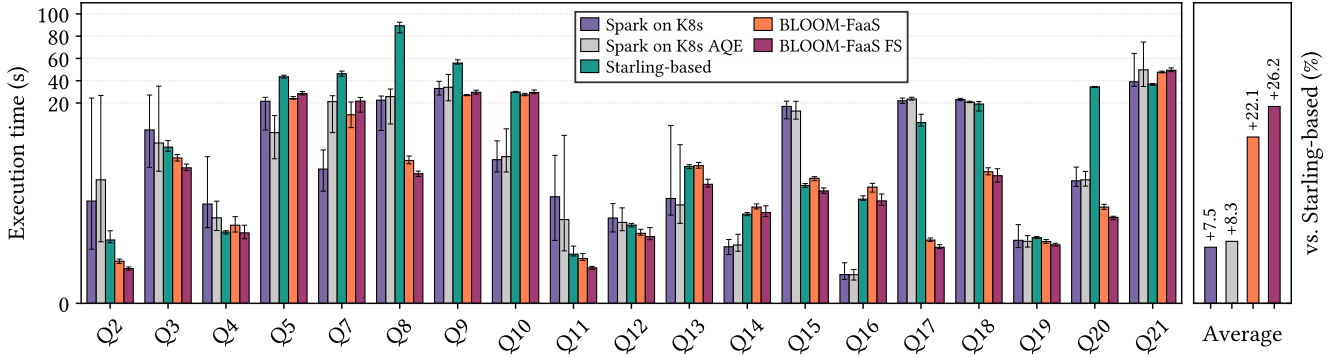


Fig. 6: Per-query execution time on TPC-H Scale Factor 100. Bars show average execution time (seconds) for five systems across 19 queries; error bars indicate min-max over runs. The right panel shows the average change relative to the Starling-based.

can be optimized by early filtering with BFs, and (2) tasks that cannot be optimized (The main idea based on Amdahl’s law [8]).

Table II presents the results of applying Amdahl-based analysis to 19 TPC-H queries at scale factor 100. For each query, the total task execution time (the sum of the individual execution times of all functions) is divided into two components: Fixed, representing steps that cannot be optimized by BFs, and Optimizable, representing steps that can be optimized by BFs. The ratio $p = \frac{\text{Optimizable}}{\text{Fixed} + \text{Optimizable}}$ represents the ratio of the execution time that can be improved by applying the BF. The BLOOM-FaaS columns report the actual Fixed/Optimizable values after early filtering, along with the number of BFs constructed and their total size. The four Improvement columns measure reductions relative to the Starling-based baseline for S3 requests, function invocations, shuffle data volume, and total task execution time.

BLOOM-FaaS improves performance in both the Fixed and Optimizable components. For the fixed portion, the system exploits multi-threading within each function instance to parallelize scan and BF construction tasks over the same row group. As a result, scan latency is reduced even for steps that cannot be directly optimized by early filtering. For example, in Q8, Fixed time decreases from 602.7s to 381.0s (36.8% reduction); in Q9, from 638.6s to 416.6s (34.8%); and in Q17, from 346.1s to 209.7s (39.4%). These improvements primarily come from runtime optimizations inside the function instances rather than from the filtering capability of BFs themselves. For the Optimizable component, BFs create a cascading data-reduction effect throughout the execution pipeline: data filtered early in scans directly reduces the processing volume of downstream steps, such as partitioning, shuffling, and intermediate joins. For example, Q17 uses only a single 38KB BF yet reduces Optimizable time from 442.7s to 1.4s (99.7% reduction) while cutting shuffle data by 99.8%. Q20 and Q11, each using 3–4 small BFs with total size below 1MB, also achieve shuffle reductions of 99.1% and 89.7%, respectively.

The *Total BFs* column shows the number of BFs applied during the execution of each query. Q20 is a representative

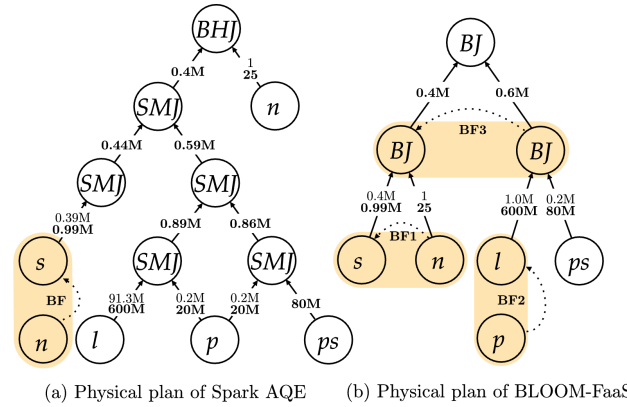


Fig. 7: Physical plans of TPC-H Q20 in Spark AQE (left) and BLOOM-FaaS (right). Spark AQE injects a single BF from *nation* to filter *supplier*, while BLOOM-FaaS constructs three BFs: BF1 ($n \rightarrow s$), BF2 ($p \rightarrow l$), and BF3 built at runtime from the intermediate join $l \bowtie ps$ and pushed down to the $s \bowtie n$ branch.

example of the benefit of constructing multiple BFs, especially BFs generated from intermediate join results. In Spark AQE, only a single BF derived from *nation* is used to filter *supplier*, while large branches such as the *lineitem* scan (600M rows) and the intermediate join chain still process massive amounts of data. In contrast, BLOOM-FaaS deploys three BFs at different points in the pipeline: BF1 filters *supplier*, and BF2, derived from *part*, directly filters *lineitem* during the scan step, reducing the number of rows from 600M to 1.0M. Most importantly, BF3 is constructed dynamically from an intermediate join result at runtime and then pushed down to downstream branches, something Spark AQE cannot support because it only injects BFs from a fixed build side before joins. As a result, the final join input is reduced to 0.4M–0.6M rows, rather than processing over 91M rows as in Spark AQE. Although the total size of the three BFs is only 0.584 MB, Q20 still achieves a 99.1% reduction in shuffle data and a 66.9% reduction in task time compared with the Starling-based

TABLE II: Comparison of BF effectiveness across TPC-H Scale Factor 100 queries. The Improvement columns report percentage reduction relative to the Starling-based baseline; positive values indicate reduced execution cost. Cells are color-coded by value: blue for the coefficient p , and a green–yellow–red gradient for improvement magnitude.

Query	Starling-based			BLOOM-FaaS				Improvement (%)			
	Fixed (s)	Optimizable (s)	p (%)	Fixed (s)	Optimizable (s)	Total BFs	BFs size (MB)	S3 requests	Invocations	Shuffle data	Task time
Q8	602.7	3435.9	85.1	381.0	19.3	3	0.256	+93.6	+83.5	+96.3	+90.1
Q17	346.1	442.7	56.1	209.7	1.4	1	0.038	+78.5	+64.9	+99.8	+73.2
Q7	368.9	1543.1	80.7	372.1	248.0	1	0.005	+82.0	+73.3	+57.7	+67.6
Q20	309.9	471.5	60.3	255.7	2.7	4	0.584	+77.4	+38.1	+99.1	+66.9
Q9	638.6	1653.7	72.1	416.6	381.2	1	2.070	+67.7	+41.8	+72.6	+65.2
Q11	36.6	26.3	41.9	25.7	2.4	3	0.229	+73.3	+34.3	+89.7	+55.2
Q2	32.7	20.4	38.4	24.3	2.4	5	0.533	+67.5	+21.1	+93.7	+49.3
Q5	433.9	1353.5	75.7	439.0	481.4	2	0.005	+67.4	+36.5	+13.0	+48.5
Q18	277.3	281.7	50.4	213.5	77.8	1	0.103	+59.1	+35.8	+79.6	+46.8
Q3	275.2	238.4	46.4	252.5	32.6	2	15.649	+55.3	+36.5	+79.8	+44.2
Q16	20.9	22.3	51.6	21.7	7.0	1	5.661	+61.5	+21.8	+72.1	+32.9
Q10	247.2	779.9	75.9	261.4	621.4	2	7.798	+17.2	+15.3	+23.7	+13.8
Q12	153.7	19.7	11.4	165.0	2.4	1	5.567	+13.8	+5.4	+87.3	+3.2
Q19	234.1	4.6	1.9	236.0	0.7	1	0.092	−15.0	+2.2	+99.7	+0.8
Q13	184.9	86.5	31.9	190.0	86.5	0	0.0	−3.6	0.0	0.0	−1.9
Q14	225.2	9.2	3.9	235.0	7.0	1	5.014	−13.6	+0.9	+18.4	−3.5
Q4	120.6	13.8	10.2	135.5	3.2	1	10.918	+28.0	+6.7	+83.7	−3.6
Q15	219.2	16.1	6.9	229.7	17.2	1	1.808	−28.9	−0.3	0.0	−5.6
Q21	430.8	836.8	66.0	473.0	860.2	3	12.514	−5.3	−0.4	+0.3	−5.6

baseline.

Although most BFs are small (median 0.58 MB, with 14/18 queries below 1 MB), constructing and merging BFs in a serverless environment still introduces significant fixed overhead. When the optimizable ratio p is low, this overhead can outweigh the benefits. Q19 ($p = 1.9\%$) is an example: despite reducing shuffle data by 99.7%, task time improves by only 0.8%, while S3 requests increase by 15.0% due to the overhead of constructing and distributing BFs. Q14 ($p = 3.9\%$) and Q15 ($p = 6.9\%$) exhibit similar behavior, with task time degrading by 3.5% and 5.6%, respectively. Q21 represents the worst case: the upper bound on p is 66.0%, the query constructs three BFs totaling 12.5 MB over a self-join structure but fails to significantly reduce downstream data volume (shuffle decreases by only 0.3%), resulting in a 5.6% increase in task time.

D. Self-Calibration Analysis

We evaluate BLOOM-FaaS self-calibration against static BFs with a target false positive rate of 0.001. We consider only queries whose build sides exceed 10M rows, and report average observed element counts for repeated executions. Overall, self-calibration reduces inserted elements by 95.85%. The largest reductions occur when predicates substantially shrink the build input, as in Q15, Q17, Q19, and Q20. For example, Q17 reduces the BF input from 20M to only 22K rows. Q21 benefits less because its self-joins retain a large runtime cardinality, requiring larger BFs and limiting pruning effectiveness.

E. Object Storage Overhead Analysis

This section evaluates the impact of the two-tier storage architecture combined with early filtering on shuffle data

TABLE III: BF element-count reduction from runtime self-calibration on TPC-H Scale Factor 100. Element counts are summed across all BF-building sources per query.

Query	Base-table cardinality	Self-calibration	Ratio	Reduction
Q3	165.00M	9.13M	18.1×	94.47%
Q4	150.00M	6.37M	23.5×	95.75%
Q10	150.00M	836.87K	179.2×	99.44%
Q12	600.04M	3.25M	184.7×	99.46%
Q14	600.04M	3.78M	158.9×	99.37%
Q15	600.04M	1.09M	548.9×	99.82%
Q17	20.00M	22.26K	898.6×	99.89%
Q19	20.00M	53.50K	373.9×	99.73%
Q20	101.00M	340.76K	296.4×	99.66%
Q21	151.00M	81.19M	1.9×	46.23%

costs and analyzes the effect of different compression codecs (NONE, SNAPPY, GZIP, and ZSTD [41]) on overall system performance. We consider four primary metrics: intermediate data size, function invocations, processing time, and total S3 requests (GET, PUT, and LIST/HEAD).

The two-tier storage architecture of BLOOM-FaaS consists of two distinct cost models: long-term storage is charged based on the number of requests, while short-term storage is charged according to the amount of I/O data transferred. Using the same compression codec, BLOOM-FaaS with early filtering reduces the total number of S3 requests by 2.6–2.7× on the long-term tier and by 8.2–12.6× on the short-term tier compared with the Starling-based system. When BLOOM-FaaS FS is used as the intermediate data exchange layer, all BF

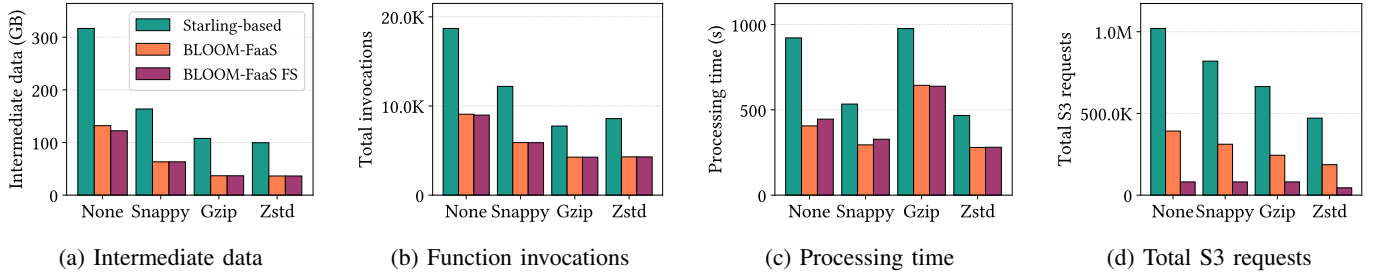


Fig. 8: Performance and resource usage of Starling-based, BLOOM-FaaS, and BLOOM-FaaS FS across four Parquet codecs on TPC-H Scale Factor 100. All values are summed across the 19 queries and averaged per run.

and shuffle traffic bypasses S3 entirely, reducing the number of PUT requests per run from 32K–84K to fewer than 10 since only the final query output must be written to long-term storage (See Figure 8).

Compression codecs also significantly affect overall performance. On the long-term tier, GZIP reduces the total number of requests by approximately 35% compared with the uncompressed configuration (from 1.02M to 664K), due to its stronger compression ratio and the resulting reduction in GET requests during scan operations. However, Gzip incurs substantially higher CPU overhead, resulting in a total processing time of 638.56s on BLOOM-FaaS FS, which is approximately 2.27× slower than ZSTD under the same configuration (280.84s).

Among the four evaluated configurations, ZSTD combined with FS achieves the best balance between data size, processing time, and CPU overhead. Compared with Snappy, Zstd further reduces data size by 42.5% (from 63.30 GB to 36.38 GB) while also reducing processing time by approximately 14.3% (from 327.89s to 280.84s).

F. Estimate Monetary Cost Analysis

This section compares the monetary cost of BLOOM-FaaS with the Starling-based baseline and two AWS EMR deployment models. We estimate the monetary cost of each system using the resource-consumption metrics collected during our experiments, including total FaaS execution time, the number of S3 file accesses, and the volume of intermediate data. We consider five configurations: BLOOM-FaaS with S3, BLOOM-FaaS with NFS (EFS Bursting), the Starling-based baseline, AWS EMR Serverless, and AWS EMR on EKS.

For BLOOM-FaaS and the Starling-based baseline, the estimated monetary cost is computed using Equation 7. Here, C_{cpt} denotes the function compute cost, C_{inv} the invocation cost, C_{eph} the function’s ephemeral storage cost, C_{obj} the object storage cost, and C_{fs} the shared file system cost. For the AWS EMR configurations, we use Equation 8, where C_{res} represents the cost of resource consumption, C_{infra} the cost of provisioned infrastructure, and C_{obj} the object storage cost. Detailed cost calculations and pricing assumptions are provided in Appendices A and B.

$$C_{Serverless}(q) = C_{cpt} + C_{inv} + C_{eph} + C_{obj} + C_{fs} \quad (7)$$

$$C_{SparkEMR}(q) = C_{res} + C_{infra} + C_{obj} \quad (8)$$

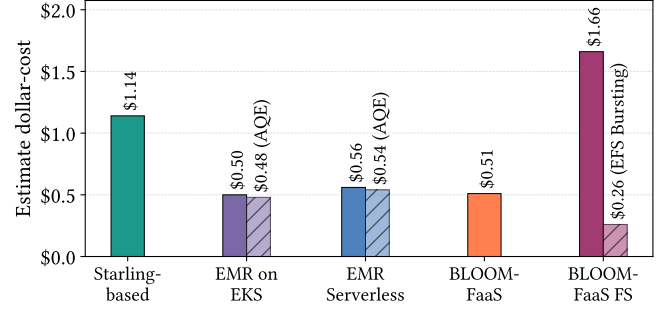


Fig. 9: Estimated monetary cost of five systems using official AWS pricing, excluding Free Tier discounts (US East (N. Virginia), August 2026).

Figure 9 presents the estimated monetary cost of the five systems based on official AWS pricing for the TPC-H workload with a scale factor of 100. BLOOM-FaaS with S3 reduces the monetary cost by 55.3% compared with the Starling-based baseline and by 8.9% compared with AWS EMR Serverless, while its cost is only 2.0% higher than that of AWS EMR on EKS. BLOOM-FaaS FS using EFS Bursting achieves even greater cost savings, reducing the cost by 77.3% compared with the Starling-based baseline, by 48.0% compared with AWS EMR on EKS, and by 53.6% compared with AWS EMR Serverless.

These results demonstrate the cost efficiency of serverless data processing, particularly for workloads that are executed infrequently. In addition to eliminating the need for cluster provisioning, configuration, and management, BLOOM-FaaS is less affected by cluster startup overhead than AWS EMR on EKS. Therefore, the serverless execution model provides advantages in terms of both monetary cost and operational simplicity while maintaining competitive query-processing performance.

VI. CONCLUSION

In this paper, we present BLOOM-FaaS, a serverless join framework that leverages distributed early filtering within FaaS constraints. The framework builds filters at multiple levels via fan-out/fan-in over short-term storage, with an adaptive coordinator that self-calibrates filter size and downstream parallelism from metadata feedback, catalog-less. On 330 queries randomly generated from TPC-H query templates at Scale Factor 100, BLOOM-FaaS reduces intermediate data by 63.46%, function invocations by 51.1%, and total object storage requests by 42.5%, improving execution time by up to 26.2% and overall monetary cost by 55.3% over a Starling-based baseline.

Future work. Several directions follow naturally. First, a cost-based rule could apply early filtering only when both high selectivity and partition-elimination conditions hold. Second, evaluate the system under data skew, a common characteristic of real-world workloads. Finally, combining LLMs with retrieval-augmented generation could let users describe pipelines in natural language, with the system synthesizing FaaS DAGs tuned to a target objective, such as processing time or service cost.

VII. ACKNOWLEDGMENT

Experiments presented in this paper were carried out using the Grid'5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER and several Universities as well as other organizations (see <https://www.grid5000.fr>).

REFERENCES

- [1] Amazon Web Services: Amazon elastic file system (efs) (2026), <https://aws.amazon.com/efs/>, accessed: 2026-01-02
- [2] Amazon Web Services: Amazon elasticache (2026), <https://aws.amazon.com/elasticache/>, accessed: 2026-01-02
- [3] Amazon Web Services: Amazon simple storage service (s3) (2026), <https://aws.amazon.com/s3/>, accessed: 2026-01-02
- [4] Amazon Web Services: Big data platform - amazon emr (2026), <https://aws.amazon.com/emr/>, accessed: 2026-01-03
- [5] Amazon Web Services: Fast nosql key-value database - amazon dynamodb - aws (2026), <https://aws.amazon.com/dynamodb/>, accessed: 2026-01-02
- [6] Amazon Web Services: Interactive sql - serverless query service - amazon athena (2026), <https://aws.amazon.com/athena/>, accessed: 2026-01-03
- [7] Amazon Web Services: Serverless function, faas serverless - aws lambda (2026), <https://aws.amazon.com/lambda/>, accessed: 2026-01-03
- [8] Amdahl, G.M.: Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS '67 (Spring) (1967)
- [9] Apache Software Foundation: Orc files (2026), <https://spark.apache.org/docs/latest/sql-data-sources-orc.html>, accessed: 2026-08-19
- [10] Apache Software Foundation: Parquet bloom filter (2026), <https://parquet.apache.org/docs/file-format/bloomfilter/>, accessed: 2026-08-19
- [11] Balouek, D., Carpen Amarie, A., Charrier, G., Desprez, F., Jeannot, E., Jeanvoine, E., Lèbre, A., Margery, D., Niclausse, N., Nussbaum, L., Richard, O., Pérez, C., Quesnel, F., Rohr, C., Sarzyniec, L.: Adding virtualization capabilities to the Grid'5000 testbed. In: Cloud Computing and Services Science, pp. 3–20. Springer International Publishing (2025)
- [12] Barcelona-Pons, D., Arjona, A., García-López, P., Molina-Giménez, E., Klymonchuk, S.: Burst computing: quick, sudden, massively parallel processing on serverless resources. In: Proceedings of the 2025 USENIX Conference on Unix Annual Technical Conference. USENIX Association (2025)
- [13] Bian, H., Geng, D., Chai, Y., Ailamaki, A.: Serverless query processing with flexible performance slas and prices (2024)
- [14] Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM* **13**, 422–426 (1970)
- [15] Bodner, T., Radig, T., Justen, D., Ritter, D., Rabl, T.: An empirical evaluation of serverless cloud infrastructure for large-scale data processing. In: International Conference on Extending Database Technology (2025)
- [16] Bratbergsengen, K.: Hashing methods and relational algebra operations. In: Proceedings of the 10th International Conference on Very Large Data Bases. Morgan Kaufmann Publishers Inc. (1984)
- [17] Broder, A., Mitzenmacher, M.: Network Applications of Bloom Filters: A Survey. *Internet Mathematics* (2004)
- [18] Christensen, K., Roginsky, A., Jimeno, M.: A new analysis of the false positive rate of a bloom filter **110**(21), 944–949 (2010)
- [19] Eismann, S., Bui, L., Grohmann, J., Abad, C., Herbst, N., Kounev, S.: Sizeless: predicting the optimal size of serverless functions. Association for Computing Machinery (2021)
- [20] Google Cloud Platform: Bigquery — ai data platform — lakehouse — edw (2026), <https://cloud.google.com/bigquery>, accessed: 2026-01-03
- [21] Google Cloud Platform: Cloud run functions (2026), <https://cloud.google.com/functions>, accessed: 2026-01-03
- [22] Google Cloud Platform: Firestore (2026), <https://cloud.google.com/products/firestore>, accessed: 2026-01-02
- [23] Google Cloud Platform: Memorystore: in-memory redis compatible data store (2026), <https://cloud.google.com/memorystore>, accessed: 2026-01-02
- [24] Khandelwal, A., Kejariwal, A., Ramasamy, K.: Le Taureau: Deconstructing the Serverless Landscape & A Look Forward. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. pp. 2641–2650 (2020)
- [25] Khandelwal, A., Tang, Y., Agarwal, R., Akella, A., Stoica, I.: Jiffy: Elastic far-memory for stateful serverless analytics. In: Proceedings of the Seventeenth European Conference on Computer Systems (2022)
- [26] Klimovic, A., Wang, Y., Stuedi, P., Trivedi, A., Pfefferle, J., Kozyrak, C.: Pocket: Elastic ephemeral storage for serverless analytics. In: Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation. pp. 427–444. USENIX Association (2018)
- [27] Koutsoukos, D., Marroquín, R., Müller, I., Klimovic, A.: Adaptive data transformations for QaaS. Conference on Innovative Data Systems Research (CIDR) (2025)
- [28] Koutsoukos, D., Marroquín, R., Müller, I., Klimovic, A.: Adaptive data transformations for qaas
- [29] Lin, C., Khazaei, H.: Modeling and optimization of performance and cost of serverless applications (2021)
- [30] Mackert, L.F., Lohman, G.M.: R* optimizer validation and performance evaluation for distributed queries. In: Proceedings of the 12th International Conference on Very Large Data Bases (1986)
- [31] Microsoft Azure: Azure blob storage: Massively scalable and secure object storage for cloud-native workloads, archives, data lakes, high-performance computing, and machine learning (2026), <https://azure.microsoft.com/products/storage/blobs/>, accessed: 2026-01-02
- [32] Microsoft Azure: Azure files (2026), <https://azure.microsoft.com/en-us/products/storage/files>, accessed: 2026-01-02
- [33] Microsoft Azure: Azure functions (2026), <https://azure.microsoft.com/en-us/products/functions>, accessed: 2026-01-02
- [34] Molina-Giménez, E., Barcelona-Pons, D., Iaconopelli, O.H., García-López, P.: Flexecutor: Out-of-the-box smart provisioning for serverless workflows. In: Proceedings of the 11th International Workshop on Serverless Computing (2025)
- [35] Müller, I., Marroquín, R., Alonso, G.: Lambda: Interactive Data Analytics on Cold Data Using Serverless Cloud Infrastructure. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. pp. 115–130. ACM (2020)
- [36] Perron, M., Castro Fernandez, R., DeWitt, D., Madden, S.: Starling: A Scalable Query Engine on Cloud Functions. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. pp. 131–141. ACM
- [37] Phan, T.C., d'Orazio, L., Rigaux, P.: Toward intersection filter-based optimization for joins in mapreduce. In: Proceedings of the 2nd International Workshop on Cloud Intelligence. ACM
- [38] Rababa, S., Al-badarnah, A.: Optimizations for filter-based join algorithms in mapreduce. *Journal of Intelligent & Fuzzy Systems: Applications in Engineering and Technology* pp. 8963–8980 (2021)

- [39] Sreekanti, V., Wu, C., Lin, X.C., Schleier-Smith, J., Gonzalez, J.E., Hellerstein, J.M., Tumanov, A.: Cloudburst: Stateful functions-as-a-service. pp. 2438–2452
- [40] Yu, X., Youill, M., Woicik, M., Ghanem, A., Serafini, M., Aboulmaga, A., Stonebraker, M.: PushdownDB: Accelerating a DBMS using S3 Computation (Feb 2020)
- [41] Zhu, C., Han, B., Li, G.: PAC: A monitoring framework for performance analysis of compression algorithms in Spark

APPENDIX

This appendix explains how to estimate the dollar cost using information obtained from the demonstration.

APPENDIX A

MONETARY COST ESTIMATE MODEL FOR FAAS-BASED SYSTEMS

This model applies to BLOOM-FaaS, BLOOM-FaaS FS, and the Starling-based baseline. All three are decomposed into five additive components $C_{\text{Serverless}}(q) = C_{\text{cpt}} + C_{\text{inv}} + C_{\text{eph}} + C_{\text{obj}} + C_{\text{fs}}$

A. Function compute cost

The compute cost of FaaS execution is estimated based on the total billed GB-seconds consumed by all function invocations:

$$C_{\text{cpt}} = \pi_{\text{gbs}} \sum_{s \in S} n_s M_s \bar{d}_s \quad (9)$$

where π_{gbs} denotes the AWS Lambda price per GB-second, S is the set of function stages, n_s is the number of function invocations at stage s , M_s is the memory allocated to each function (in GB), and $\bar{d}_s$ is the average billed execution duration (in seconds) at stage s . Thus, $n_s M_s \bar{d}_s$ represents the total billed GB-seconds consumed by stage s .

B. Invocation cost

In addition to compute charges, AWS Lambda incurs a request-based cost for each function invocation. We estimate the total invocation cost as:

$$C_{\text{inv}} = \pi_{\text{req}} \sum_{s \in S} n_s \quad (10)$$

where π_{req} denotes the AWS Lambda invocation price per request and n_s is the number of function invocations at stage s .

C. Ephemeral storage cost of Function

AWS Lambda provides 0.5 GB of ephemeral storage per function invocation at no additional cost, only the amount of ephemeral storage exceeding this free allocation is billed. We estimate the total ephemeral storage cost as:

$$C_{\text{eph}} = \pi_{\text{eph}} \sum_{s \in S} n_s \bar{d}_s \max(0, E_s - 0.5) \quad (11)$$

where π_{eph} denotes the AWS Lambda ephemeral storage price per GB-second, n_s is the number of function invocations at stage s , E_s is ephemeral storage configured at stage s .

D. Object storage cost

The object storage cost consists of two components: the cost of S3 requests and the cost of temporarily storing intermediate data during data exchanges. We estimate the total object storage cost as:

$$C_{\text{obj}} = \sum_{o \in O} N^o \pi_o + V_{S3} \pi_{S3} \frac{T_{\text{hold}}}{730 \text{ h}} \quad (12)$$

where O denotes the set of S3 request classes (e.g., GET and PUT), N^o is the number of requests of class o , and π_o is the corresponding price per request. V_{S3} denotes the volume of intermediate data temporarily stored in S3 (in GB), while π_{S3} is the S3 storage price per GB-month and $\frac{T_{\text{hold}}}{730 \text{ h}}$ is the duration for which intermediate data is retained in S3 during the shuffle exchange, and 730 hours represents the average number of hours in one month.

E. Shared file system cost

The shared file system cost consists of four components: temporary storage, data reads, data writes, and cross-Availability Zone (AZ) data transfer. We estimate the total file system cost as:

$$C_{\text{fs}} = V_{\text{fs}} \pi_{\text{fs}} \tau + R_{\text{fs}} \pi_{\text{rd}} + W_{\text{fs}} \pi_{\text{wr}} + X_{\text{AZ}} \pi_{\text{az}} \quad (13)$$

where V_{fs} denotes the volume of intermediate data temporarily stored in Amazon EFS, including local BF, global BF, and other intermediate data structures. π_{efs} denotes the EFS storage price per GB-month, while τ is the retention factor that converts the temporary storage duration into a fraction of a billing month. R_{efs} and W_{efs} denote the total volumes of data read from and written to EFS, respectively, while π_{rd} and π_{wr} denote their corresponding per-GB charges. Finally, X_{AZ} denotes the volume of data transferred across Availability Zones, and π_{az} represents the corresponding cross-AZ data transfer price.

APPENDIX B

MONETARY COST ESTIMATE MODEL FOR SPARK SERVERLESS

This model applies to BLOOM-FaaS, BLOOM-FaaS FS, and the Starling-based baseline. Both are decomposed into three additive components $C_{\text{SparkEMR}}(q) = C_{\text{res}} + C_{\text{infra}} + C_{\text{obj}}$

A. Resource consumption cost

For AWS EMR Serverless, the resource consumption cost is determined by the provisioned vCPU, memory, and ephemeral storage resources. Let $R = \{v, m, e\}$ denote the set of resource types corresponding to vCPU, memory, and ephemeral storage, respectively. Since each resource type is billed independently, the total resource consumption cost is estimated as:

$$C_{\text{res}} = \sum_{r \in R} Q_r T_r \pi_r \quad (14)$$

where Q_r denotes the amount of resource r provisioned for the job, $T_r = \max(\frac{1}{60} \text{ h}, T_{\text{job}})$ is the corresponding billed duration in hours, and π_r denotes the unit price of resource r per unit-hour. Specifically, Q_v , Q_m , and Q_e represent the provisioned vCPU count, memory capacity, and ephemeral storage capacity, respectively, while π_v , π_m , and π_e denote their corresponding unit prices.

B. Provisioned infrastructure cost

AWS EMR on EKS and AWS EMR Serverless follow different infrastructure provisioning models. With EMR on EKS, users must maintain the underlying EKS cluster and worker nodes during the entire provisioning period. In contrast, EMR Serverless does not require users to provision or manage worker nodes or a Kubernetes control plane.

For EMR on EKS, the infrastructure cost consists of EC2 worker instances, the EKS control plane, and EBS volumes attached to the worker nodes. We estimate this cost as:

$$C_{\text{infra}}^{\text{EKS}} = (N_{\text{node}}\pi_{\text{inst}} + \pi_{\text{k8s}})T_{\text{prov}} + N_{\text{node}}V_{\text{EBS}}\pi_{\text{EBS}}\frac{T_{\text{prov}}}{730 \text{ h}}, \quad (15)$$

where N_{node} is the number of worker nodes, π_{inst} the EC2 price per node-hour, π_{k8s} the control-plane price per hour, and V_{EBS} the per-node storage in GB priced at π_{EBS} per GB-month, scaled by the provisioning duration as a fraction of a 730-hour billing month. Since T_{prov} spans startup, execution, and teardown, the user is billed for a strictly longer interval than the query occupies.

For AWS EMR Serverless, the underlying compute infrastructure is fully managed by AWS. Users do not explicitly provision EC2 worker nodes, EBS volumes, or an EKS control plane. Consequently, the user-visible provisioned infrastructure cost is:

$$C_{\text{infra}}^{\text{Serverless}} = 0. \quad (16)$$

C. Object storage cost

Because Spark provides a built-in data exchange mechanism during query execution, its object storage cost follows the same formulation as Eq. 12. However, this cost is lower because object storage is primarily used for reading input data and storing final query results, rather than for exchanging intermediate data between processing stages.